

Do We Still Need to Be Agile? What Two Cohorts of Student Projects Reveal About Planning Discipline in the GenAI-Era

Abstract—V9 abstract content will be written after the approved Methodology, Results, Discussion, Threats to Validity, and Conclusion sections.

I. INTRODUCTION

V9 introduction content will be revised after the results catalog and approved section-level evidence are available.

II. BACKGROUND AND RELATED WORK

V9 background and related-work content will be aligned with the approved empirical definitions during the editorial phase.

III. METHODOLOGY

To investigate how planning discipline and coordination evolve in AI-assisted software engineering projects, we designed an empirical study combining longitudinal repository mining, standardized instructor evaluations, multi-checkpoint student surveys, and qualitative meeting transcripts across two academic offerings.

A. Context, Course Design, and “Rules of the Game”

The study was conducted within a project-based undergraduate software engineering course at the State University of Ceará (UECE), spanning two semesters: 2025.2 ($N = 9$ analyzable teams) and 2026.1 ($N = 5$ teams), yielding a total inferential dataset of $N = 14$ team-semesters (≈ 70 participating students).

Students formed teams of approximately five members through spontaneous self-allocation, with remaining students assigned to vacant slots to ensure balanced team sizes. Teams enjoyed complete autonomy regarding their internal organization, process models, and project management methodologies (e.g., choosing freely between Agile variants, Scrum, Kanban, or unguided approaches). However, course guidelines enforced two structural constraints:

- 1) Teams were required to develop an operational software project featuring an integrated Generative AI component as a core functional requirement. Specifically, projects were expected to implement advanced architectural patterns such as Retrieval-Augmented Generation (RAG)—a technique that dynamically grounds Large Language Models (LLMs) in external knowledge bases to mitigate hallucination [1]—or LLM-driven data analytics pipelines.

- 2) Students were strongly encouraged to utilize generative AI assistants throughout all phases of software development, from requirements specification to implementation and testing.

The project lifecycle was anchored by three synchronized temporal checkpoints (T_1 , T_2 , and T_3) paced identically across both cohorts:

- T_1 (**Baseline**): Administered on the first day of class prior to any formal instruction on generative AI processes, capturing baseline perceptions, initial project scope, and early planning commits.
- T_2 (**Mid-Semester**): Conducted following structured instructional modules covering generative AI mechanics, RAG architectures, and software process models, accompanied by the second wave of survey instruments and status presentations.
- T_3 (**Final Integration**): Positioned one week prior to the final project delivery and presentation, capturing the peak pressure of system integration and deadline convergence.

B. Project Scope and Real-World Complexity

Course guidelines prohibited trivial algorithmic exercises and required teams to address community, educational, or business problems. The resulting systems covered healthcare, finance, academic research, education, and ecotourism, while embedding Generative AI into functional architectures. This constraint was retained because it provides the empirical context for interpreting repository change, planning evidence, and evaluator outcomes.

C. Data Sources and Collection Framework

Our framework triangulates four evidentiary channels.

1. Git repositories: read-only parent mirrors and the canonical lake contracts provide commit, file-change, timestamp, author-hash, and path- provenance observations. Git activity is treated as repository evidence, not as telemetry of AI-tool use or a complete record of work outside the repository.

2. Independent evaluator scorecards: teams presented at T_1 , T_2 , and T_3 . Independent evaluators submitted private scores for project progress, scope applicability, technical complexity, and engagement/ participation. The canonical evaluator contract is team-semester by checkpoint.

3. Multi-checkpoint student surveys: students completed survey instruments at all three checkpoints. They covered

TABLE I
ANALYZED STUDENT PROJECTS BY COHORT AND DOMAIN

Semester	Project	System description and AI integration scope
2026.1	FoodGuard	Web platform using nutritional data and medical history to identify food allergens.
2026.1	RevisAI	Educational system generating flashcards and customized study cycles.
2026.1	SmartFlowIA	WhatsApp chatbot for university-freshman onboarding and orientation.
2026.1	Interview Prep IA	Resume analysis, interview simulation, and candidate feedback.
2026.1	EnemBot	Domain-specific chatbot for Brazilian National High School Exam tutoring.
2025.2	UECEncontra	Virtual assistant and campus-navigation system for university resources.
2025.2	EstudaAI	Multimodal study-management platform for summaries, reminders, and grades.
2025.2	CareerPath IA	Resume analysis and targeted skill-development recommendations.
2025.2	Buliçoso	Medical-leaflet simplification and medication reminders for older adults.
2025.2	EcoRoteiro	Personalized ecotourism itineraries based on user profiles and constraints.
2025.2	Research Flow	Academic research assistant for article search, summaries, and formatting.
2025.2	ExamForge	Exam simulator generating assessments from prior materials.
2025.2	SoFiA	Financial-management platform for university students.
2025.2	Synapse	Requirements tool generating requirements, user stories, and backlogs.

demographics, professional experience, Generative AI adoption, perceived role impact, and project expectations. Survey responses are cohort-by-semester/checkpoint data and do not carry team identifiers; M1 and M2 therefore remain at that grain.

4. Qualitative transcripts: anonymized recordings and supervisory discussions were segmented into 100 technical transcript chunks for 2025.2 (35 at T_1 , 37 at T_2 , and 28 at T_3). M5 reconstructs one global corpus per checkpoint; chunks and source sessions are provenance units, not independent observations. Transcript coverage for 2026.1 is unavailable and is never imputed.

Data anonymization and privacy: raw streams were sanitized before analysis using deterministic masking and LLM-assisted redaction. Finalized analytical artifacts exclude direct personal identifiers. LLM ledger and raw payload provenance remain restricted to the internal reproducibility record.

D. Advanced Metrics Operationalization

The metrics are organized into nine families, preserving the study’s conceptual map while allowing each family to expose multiple non-composite components. Each component has an explicit analytical grain, source contract, missingness state, and artifact reference. No component is silently combined into a new score.

RQ1 families. M1 (perception and self-reported usage panel) separates perception families, usage frequencies, tasks, tools, prior experience, autonomy/tool balance, and exploratory career topics. M2 (role perception) preserves role-specific Likert distributions and privacy-preserving paired sensitivity summaries. Both are self-report measures, not AI-use telemetry.

RQ2 families. M3 (author activity dynamics) separates phase activity, author concentration, and rolling activity, including a pooled relative-window view. M4 (clean change dynamics) separates clean-change magnitude, intensity, artifact composition, and rolling clean churn under the central `code-churn-metrics-v2` policy. M5 (coordination evidence) reports lexical marker density, subtype composition, corpus coverage, and an auditable source chunk queue; it

remains at global corpus-by-checkpoint grain and is not correlated with team-level metrics.

RQ3 families. M6 separates structural T1 planning evidence (M6a) from structured declarative content (M6b), with M6b categories and literal evidence quotes kept separate and no composite score. M7 measures recent repository inactivity, not planning omission. M8 separates clean rework magnitude, ratio, rolling trajectory, and baseline eligibility. M9 reports descriptive, stratified associations and leave-one-out diagnostics; M7 is not used as an independent predictor.

E. LLM Usage in Data Extraction, Processing, and Metrics

All LLM calls route through one centralized gateway and are persisted in the internal structured ledger. Text tasks use `gpt-4o-mini` at temperature zero; audio transcription uses `whisper-1`. The paper-facing methodology reports only the role of these processes, while full prompts, payload hashes, raw responses, costs, and cache keys remain in the internal provenance artifacts.

M1 uses already-produced survey coding. M5 V9 uses deterministic lexical coding of the transcript corpus and issues no new LLM calls. M6b uses structured extraction from T1 commit subjects for nine observed team-semesters; five team-semesters have no T1 subjects and remain `unavailable_not_measured`. Human review approved M6b for exploratory use, with literal evidence quotes and separate categories for goals, architecture/design, task decomposition, and risk/dependency. M6b is not an architectural-quality score.

Persistence and auditability. The internal ledger records correlation identifiers, operation/model, prompts and responses, hashes, token usage, duration, estimated cost, and status. This supports internal tracing without exposing pipeline history in the article narrative.

Inference boundary. All reported metric families and components are descriptive or exploratory. Missing evidence remains distinct from zero or low quality, incompatible analytical grains are not silently combined, and no causal claim is made without a design that supports it.

IV. RESULTS

This scaffold reserves the modular Results section. Quantitative claims will be sourced exclusively from `results_summary.json` and approved section-level figures or ad hoc analyses.

V. DISCUSSION

Discussion content will interpret the approved descriptive and exploratory results without causal overreach.

VI. THREATS TO VALIDITY

Threats to validity will document grain differences, Git coverage, path classification, transcript lexical coding, LLM review, sample size, and influential cases.

VII. CONCLUSION

The conclusion will be written only after the preceding editorial sections have been approved.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [2] A. Nguyen-Duc *et al.*, “A systematic literature review on generative ai in software engineering,” *Preprints*, May 2026. [Online]. Available: <https://www.preprints.org/manuscript/202605.1638>
- [3] IEEE Computer Society, “Transforming software engineering processes through generative ai,” *IEEE Computer*, July 2025. [Online]. Available: <https://www.computer.org/csdl/magazine/co/2025/07/11052608/27RKcQkdNg4>
- [4] Cambridge Advance Online, “The contextual gap in genai for software engineering,” July 2026. [Online]. Available: <https://advanceonline.cam.ac.uk/blog/the-contextual-gap-in-genai-for-software-engineering>
- [5] GitHub Blog, “Spec-driven development with ai: Get started with a new open source toolkit,” September 2025. [Online]. Available: <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>
- [6] Thoughtworks, “Spec-driven development,” 2025. [Online]. Available: <https://www.thoughtworks.com/en-br/insights/decoder/s/spec-driven-development>
- [7] Codebridge, “The hidden costs of ai-generated software: Why ‘it works’ isn’t enough,” February 2026. [Online]. Available: <https://www.codebridge.tech/articles/the-hidden-costs-of-ai-generated-software-why-it-works-isnt-enough>
- [8] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, “Manifesto for agile software development,” 2001. [Online]. Available: <https://agilemanifesto.org/>
- [9] W. Cunningham, “The wycash portfolio management system,” in *OOP-SLA ’92 Experience Report*. ACM, 1992.
- [10] M. Fowler, “Technical debt quadrant,” 2009. [Online]. Available: <https://martinfowler.com/bliki/TechnicalDebtQuadrant.html>
- [11] P. Kruchten, R. L. Nord, and I. Ozkaya, “Technical debt: From metaphor to theory and practice,” *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012.
- [12] F. P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, 1975.
- [13] T. DeMarco and T. Lister, *Peopleware: Productive Projects and Teams*. Dorset House, 1987.
- [14] Z. Li, P. Avgeriou, and P. Liang, “A systematic mapping study on technical debt and its management,” *Journal of Systems and Software*, vol. 101, pp. 193–220, 2015.
- [15] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Boston, B. McGrew, S. Karthik, and et al., “Evaluating large language models trained on code,” in *arXiv preprint arXiv:2107.03374*, 2021.
- [16] C. Williams, M. Jones, and R. Patel, “Generative ai and software engineering education: Opportunities, risks, and design constraints,” in *Proceedings of the International Conference on Software Engineering Education and Training*, 2024.
- [17] S. Miller, J. Ferris, and A. Chen, “Coordination signals and human-ai teaming in software engineering courses,” in *Proceedings of the ACM/IEEE International Conference on Software Engineering*, 2023.
- [18] J. Pérez, A. Moreno, and E. Lauri, “Planning quality, rework, and team coordination in project-based software courses,” *IEEE Transactions on Education*, vol. 65, no. 2, pp. 214–223, 2022.